EJERCICIO 11

Puntos de articulación

Lenguaje natural:
Se recorre el grafo en profundidad desde un vértice raíz. Al entrar a un vértice se le asigna el siguiente número (numBp) y su bajo empieza igual que su numBp.

    Por cada vecino w:
  - si w es el padre, se ignora.
  - si w ya fue visitado, la arista es de retroceso, y el bajo del vértice puede bajar al numBp de w.
  - si w no fue visitado: se lo recorre recursivamente y al volver el bajo del vértice puede bajar al bajo del hijo. Además, si bajo[hijo] >= numBp[vértice], el hijo y todo lo que cuelga de él solo se conectan al resto pasando por este vértice, así que es un punto de articulación a no ser que sea la raíz.

Para la raíz se cuenta cuántos hijos tiene en el árbol de la búsqueda: con 2 o más, es punto.
Si el grafo no es conexo se repite la búsqueda desde cada vértice que todavía no se visitó, así se obtienen los puntos de cada componente.

Precondiciones:
- El grafo no es nulo.
- Es no dirigido.
- No tiene lazos ni aristas repetidas entre el mismo par de vértices.
- Es conexo, si no lo es, se devuelven los puntos de cada componente.

Postcondiciones:
- El grafo no cambia.
- Se devuelve una lista sin repetidos con los vértices que, al quitarlos, aumentan la cantidad de componentes conexas.
- La lista está vacía si no hay ninguno.

Pseudocódigo:

```
puntosDeArticulacion(grafo) devuelve Lista
Comienzo
    numBp <- diccionario vacío            // vértice -> número de orden de visita
    bajo <- diccionario vacío             // vértice -> bajo
    puntos <- lista vacía sin repetidos que conserva el orden en que se agregan

    PARA CADA raiz EN vertices(grafo) HACER
        SI raiz NO ESTA EN numBp ENTONCES
            numBp[raiz] <- cantidad de claves de numBp + 1
            bajo[raiz] <- numBp[raiz]
            hijosDeLaRaiz <- 0
            PARA CADA arista EN adyacencias(grafo, raiz) HACER
                SI arista.destino NO ESTA EN numBp ENTONCES
                    hijosDeLaRaiz <- hijosDeLaRaiz + 1
                    bpf(arista.destino, raiz)
                FIN SI
            FIN PARA
            SI hijosDeLaRaiz >= 2 ENTONCES
                puntos.agregar(raiz)
            FIN SI
        FIN SI
    FIN PARA
    devolver puntos
Fin

bpf(actual, padre)
Comienzo
    numBp[actual] <- cantidad de claves de numBp + 1
    bajo[actual] <- numBp[actual]
    PARA CADA arista EN adyacencias(grafo, actual) HACER
        w <- arista.destino
        SI w = padre ENTONCES
            continuar                     // es la arista por la que se llegó
        FIN SI
        SI w ESTA EN numBp ENTONCES
            // arista de retroceso
            bajo[actual] <- minimo(bajo[actual], numBp[w])
        SINO
```

```
            // w es un hijo
            bpf(w, actual)
            bajo[actual] <- minimo(bajo[actual], bajo[w])
            SI bajo[w] >= numBp[actual] ENTONCES
                puntos.agregar(actual)       // la raíz se trata aparte, en puntosDeArticulacion
            FIN SI
        FIN SI
    FIN PARA
Fin
```

Orden del tiempo de ejecución:
Cada vértice se visita una vez y cada arista se mira dos veces (una por extremo). Total O(V + E).